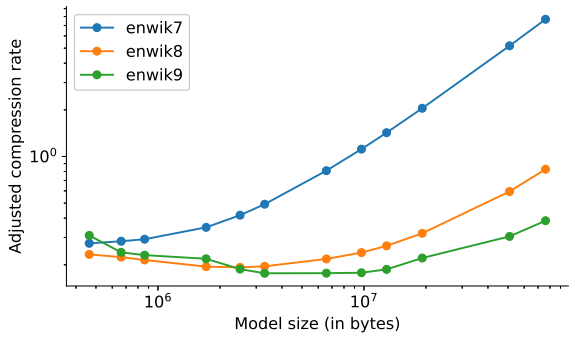


Figure 2: Adjusted compression rates (compressed size / raw size) for Transformers of different sizes, trained on enwik8 and evaluated on enwik (both axes are logarithmic). Here, the compressed size does not only consider the size of the compressed output (roughly equal to the log-loss) but also the model size, which causes all curves to increase at some point. Every dataset gives rise to an optimal model size, with a good trade-off between performance (the size of the compressed data) and cost of the model (the number of parameters). The larger the dataset, the more parameters we can afford.

Table 2: Raw compression rates (compressed size / raw size) on enwik9 for Transformers trained on enwik8 with different tokenizers, ASCII and byte-pair encoding (BPE), with various vocabulary sizes. Transformers compress better with simpler tokenizers. However, larger vocabulary sizes reduce the length of the sequence more, meaning more information can be packed into the context.

	Raw Compression Rate (%)		
Tokenizer	**200K**	**6.4M**	**38M**
ASCII	22.9	**13.6**	**6.4**
BPE 1K	25.4	14.8	6.9
BPE 2K	25.6	15.7	7.4
BPE 5K	23.1	17.1	8.7
BPE 10K	21.3	17.0	8.9
BPE 20K	**19.3**	16.4	9.0

3.4 Compressors as Generative Models

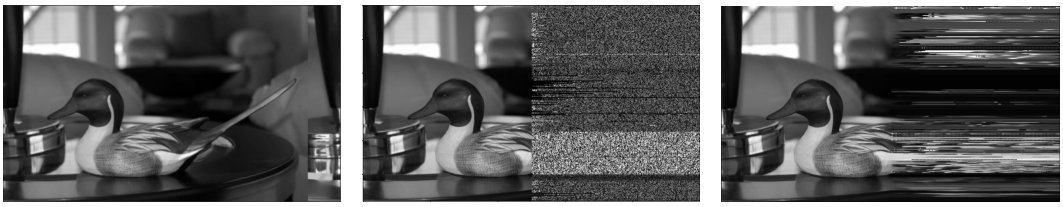
In Section 3.2 we showed that any predictor can be employed as a compressor. Here, following Section 2, we empirically demonstrate the opposite direction, i.e., that compressors can be used as a sequence prediction model, establishing our main claim that "language modeling is compression". We compute the length ℓ_c of the compressed sequence $c(x_{<i}b)$ for all possible $b \in \mathcal{X}$ to get the probabilities $\hat{\rho}(b \mid x_{<i}) = 2^{\ell_c(x_{<i}) - \ell_c(x_{<i}b)}$. This can straightforwardly be extended to sampling a whole continuation autoregressively by appending the last output to the sequence and iterating.

Theoretically, there is no strong guarantee that a good compression rate leads to "good" autoregressive samples. However, empirically it has been shown that better sequence prediction (i.e., lower log-loss) often leads to better generation (Rae et al., 2021; Brown et al., 2020). Nevertheless, in autoregressive sampling small errors often accumulate, which can lead to samples that diverge from the ground-truth distribution. Also, this standard sampling technique only looks one step into the future, and can be biased: gzip, for instance, builds an internal dictionary of 'tokens', which will be compressed using their indexes. Extending the sequence $x_{<i}$ with one of these tokens will lead to a good compression rate, but will be omitted as it can be longer than one byte. Our neural models do not suffer such bias as they are trained to predict one step ahead with the cross-entropy loss.

We compare the generative capabilities of gzip and Chinchilla 70B on images in Fig. 3. Each image is a sampled from ImageNet with height 290 and width 500. For each row in the image, we condition the model on the first 250 pixels and autoregressively generate the remaining 250 pixels, treating different rows as independent of each other (an oversimplification w.r.t. natural image statistics). We use the same byte conversions and tokenization details as explained in Appendix B. Chinchilla 70B shows signatures of visually appropriate continuations (judged qualitatively), which tend to degrade with increased sample length as more and more error accumulates. gzip produces much noisier completions. We compare the generative performance of gzip and Chinchilla (1B, 7B, and 70B) across all three data modalities in Figs. C.1 to C.3 for text, image, and audio data, respectively.

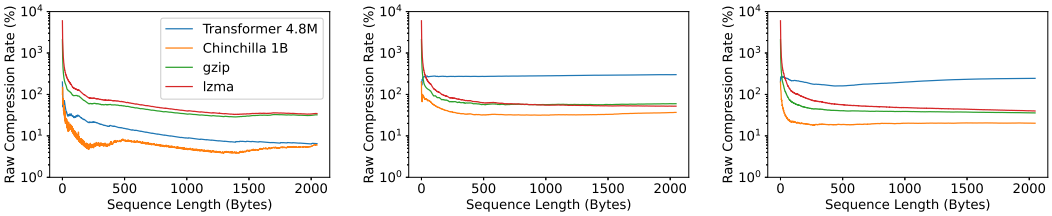
3.5 Sequential Evolution of In-Context Compression

Language models take a very different "approach" to compression compared to classical compressors. Classical compressors have a small program size and optimize for a large context length to exploit sequential dependencies in the data. In contrast, foundation models consist of billions of parameters,



(a) Original image (b) gzip (row-wise) (c) Chinchilla (row-wise)

Figure 3: Compression-based generation for image data. We condition gzip and Chinchilla on the first half of every row of the ImageNet image and then sample the remaining half autoregressively. Both models produce incoherent samples, but Chinchilla looks much less noisy than gzip.



(a) enwik9 (b) ImageNet (c) LibriSpeech

Figure 4: In-context compression rate over sequence length. For every dataset, we compute the compression rate for all subsequences of 2048 bytes, averaged over 100 sequences.

which enable rapid adaptation in their (relatively) short context window (Genewein et al., 2023). Thus, arithmetic coding-based compressors rely heavily on the predictive models' in-context learning capabilities to achieve competitive compression performance. We investigate this phenomenon in Fig. 4, which visualizes the compression rate across sequence lengths for gzip, Chinchilla 1B and a Transformer pretrained on enwik8. Intuitively, the longer the sequence, the more data the model can process in its context, and therefore, the better the compression. As expected, most compression rates decrease quickly with increasing sequence length, indicating that the models learn some data statistics in-context, without any gradient-based training. As in Table 1, the Chinchilla model achieves the best compression rates across all three data modalities and sequence lengths.

3.6 Tokenization Is Compression

Transformers are generally not trained on raw input data but on tokenized versions thereof, both for efficiency and performance reasons. As a consequence, Transformers are trained on compressed data, with tokenizers acting as the compressor. Since tokenization is known to have an impact on the generalization performance (Radford et al., 2019), we investigate its impact on the compression rate in Table 2. Concretely, we train Transformers on enwik8 using different tokenizers: ASCII, i.e., an alphabet of size 256 (no tokenization), and byte-pair encoding trained on enwik8, with various vocabulary sizes (1K, 2K, 5K, 10K, and 20K tokens). Note that the tokenizations are lossless.

Increasing the number of tokens (i.e., the "alphabet size") reduces the length of the sequence and thus increases the amount of information in a models context. However, decreasing the sequence length comes at a price: the number of tokens is larger, which makes the prediction task more challenging since reducing the entropy of the conditional distribution $\rho(x_i \mid x_{<i})$ is increasingly difficult for larger alphabet size. In theory, as the tokenization is a lossless compression, the two effects should compensate. In practice, we observe that if the model is small, increasing the number of possible tokens boosts the compression performance. In contrast, for bigger models, it seems that the converse happens: having a larger token vocabulary harms the final compression rate of the model. Nevertheless, short sequence lengths also help Transformers since their time complexity scales quadratically with context length, and it has been shown they do not generalize well to long contexts (Delétang et al., 2023; Ruoss et al., 2023).

4 RELATED WORK

Prediction vs. Compression Leveraging Shannon's source coding theorem (Shannon, 1948), a plethora of approaches exploit the connection between prediction and compression. For example, context-tree weighting (CTW) (Willems et al., 1995) mixes the predictions of many underlying Markov models to achieve lossless compression via arithmetic coding (Pasco, 1977; Rissanen, 1976). Similarly, prediction by partial matching (PPM) (Cleary & Witten, 1984) also leverages arithmetic coding, but uses a contiguous context matching method to create probability distributions based on the history of characters in a sequence. Likewise, PAQ8 (Knoll & de Freitas, 2012) uses a weighted combination of predictions from a large number of models (most of them based on context matching, but unlike PPM also noncontiguous context matches). In a different setting, Veness et al. (2015) demonstrated how to employ compression to obtain value estimates of a policy in an environment. Frank et al. (2000) and later Teahan & Harper (2003) introduced the idea of classification with compressors. Recently, Jiang et al. (2023) applied this technique with NLP tasks, paired with a k-nearest-neighbour algorithm, in which gzip achieves good results. Jiang et al. (2022) exploit the same idea but train the compressor on a vast amount of unlabeled data first. Further comparisons on the same tasks have been done between gzip and simple bag-of-words models (Opitz, 2023). Finally, van den Oord & Schrauwen (2014) apply arithmetic coding to image compression using Student distribution mixtures and Gaussian processes as predictors.

Compression With Neural Networks Prior work demonstrated that neural predictive distributions can be employed to perform lossless compression via arithmetic coding (Schmidhuber & Heil, 1994; 1996; Mahoney, 2000; Knoll, 2014; Cox, 2016; Schiopu et al., 2018; Goyal et al., 2019; Liu et al., 2019; Mentzer et al., 2019; 2020; Schiopu & Munteanu, 2020; Rhee et al., 2022; Mikolov, 2012). Similarly, neural networks were also shown to achieve strong lossless compression rates when replacing arithmetic coding with asymmetric numeral systems (Hoogeboom et al., 2019; Kingma et al., 2019; Townsend et al., 2019; Barzen et al., 2022). While these approaches assume the existence of a separate training set, a different line of work investigated arithmetic coding-based neural compression in a purely online fashion, i.e., training the model only on the data stream that is to be compressed (Bellard, 2019; Goyal et al., 2020; Bellard, 2021; Mao et al., 2022). Concurrent work (Valmeekam et al., 2023) also investigated lossless offline compression with foundation models, using arithmetic coding with Llama 2 (7B) (Touvron et al., 2023b).

Compression Biases: Tokenization, Model Size, etc. Much effort has been devoted to understanding the inductive biases of neural networks, in particular with respect to Natural Language Processing (NLP) and Transformers. Kudo & Richardson (2018) developed a tokenizer for NLP that improves over well-known techniques such as byte-pair encoding (BPE) (Sennrich et al., 2016), BPE dropout (Provilkov et al., 2020), and subword regularization (Kudo, 2018). In this paper, we show how these tokenization techniques act as pre-compressors for the data and can significantly affect the final compression rates when paired with a neural model. Other work investigated different aspects of generalization (Neyshabur et al., 2017; Mirchandani et al., 2023; Ge et al., 2023), which is equivalent to compression when accounting for the parameters' codejlength. Finally, Cheng et al. (2017) investigated compressing the neural models' parameters to further reduce the code length.

5 CONCLUSION

In this paper we investigated how and why sequence modeling is equivalent to compression. Arithmetic coding transforms a sequence model into a compressor, and, conversely, a compressor can be transformed into a predictor using its coding lengths to construct probability distributions following Shannon's entropy principle. We evaluated large language models as compressors against various standard compressors and showed that they are competitive not only on text but also on modalities they have never been trained on (image and audio data). We showed that the compression view provides novel insights on scaling laws since it takes the model size into account, unlike the log-loss objective, which is standard in current language modeling research. Consequently, we showed that the optimal model size is inextricably linked to the dataset size and cannot be scaled without limit.